

1. 题目深度解构

题意重述

给定一棵有 2^n 个点的树，点编号为 $0 \sim 2^n - 1$ 。

对每一对点 u, v ，把树临时以点

$$r = u \oplus v$$

为根，求此时 u, v 的 LCA：

$$\text{lca}_{u \oplus v}(u, v)$$

最后把所有这些 LCA 的编号按位异或起来。

要求输出：

$$\bigoplus_{0 \leq u < v < 2^n} \text{lca}_{u \oplus v}(u, v)$$

约束分析

令

$$N = 2^n$$

其中 $n \leq 21$ ，所以单组最大 $N \approx 2.1 \times 10^6$ 。

并且所有测试数据的 N 之和不超过 2^{21} 。

因此可接受复杂度大致是：

$$O(N \log N)$$

甚至最好做到：

$$O(N)$$

不能枚举点对，因为点对数量是 $O(N^2)$ 。

题目类型判定

这是一道 **图论 + XOR 组合计数 + 树上中位点性质** 的思维题。

核心不是传统 LCA 数据结构，而是利用：

- 1. 树上三点中位点；
 - 2. XOR 零三元组；
 - 3. 只关心出现次数的奇偶性。
-

2. 思维路径推导

从朴素做法开始

最直接的想法是枚举所有 u , 令 $r = u \oplus v$, 然后求:

$$\text{lca}_r(u, v)$$

但是点对有:

$$\binom{N}{2}$$

当 $N = 2^{21}$ 时完全不可行。

所以必须寻找结构。

第一个关键观察：换根 LCA 等价于三点中位点

在树上，对于任意三个点 a, b, c , 它们的三条路径:

$$P(a, b), \quad P(a, c), \quad P(b, c)$$

有唯一公共点，称为三点的 **median**，记作:

$$\text{med}(a, b, c)$$

一个经典事实是:

$$\text{lca}_r(u, v) = \text{med}(u, v, r)$$

因此本题中的每一项是:

$$\text{med}(u, v, u \oplus v)$$

第二个关键观察：点对会自然组成 XOR 为零的三元组

令:

$$w = u \oplus v$$

则:

$$u \oplus v \oplus w = 0$$

如果 u, v, w 三者互不相同且都非零，那么集合:

$$\{u, v, w\}$$

会对应三对点:

$$(u, v), \quad (u, w), \quad (v, w)$$

它们的根分别是：

$$w, \quad v, \quad u$$

而这三项的值都是同一个三点中位点：

$$\text{med}(u, v, w)$$

因为同一个值被异或了三次，等价于被异或一次。

所以，所有非退化贡献可以改写为：

对所有互不相同的非零三元组 $\{a, b, c\}$ ，满足

$$a \oplus b \oplus c = 0$$

把它们的树上中位点异或起来。

还有一种退化情况：

当 $u = 0$ 时，

$$u \oplus v = v$$

所以：

$$\text{lca}_v(0, v) = v$$

因此每个非零点 x 还会额外贡献一次 x 。

第三个关键观察：固定一个点 x ，只需看删掉 x 后的连通块大小

现在考虑一个固定点 x ，我们想知道有多少个 XOR 为零的非零三元组的中位点是 x 。

删掉点 x ，树会分成若干连通块：

$$S_1, S_2, \dots, S_k$$

对于一个三元组 $\{a, b, c\}$ ，它的中位点是 x 当且仅当：

- 如果 $x \notin \{a, b, c\}$ ，那么 a, b, c 分别落在三个不同连通块；
- 如果 $x \in \{a, b, c\}$ ，那么另外两个点落在不同连通块。

换句话说：

删掉 x 后，三元组中不等于 x 的点，不能有两个在同一个连通块里。

Aha! Moment: XOR 条件突然消失了

我们只关心奇偶性。

设所有非零 XOR 零三元组总数为 T 。

对于固定点 x ，先假设所有三元组都贡献 x 。然后把“不合法”的三元组翻转掉。

一个三元组不合法，当且仅当存在两个点在 $T - x$ 的同一个连通块中。

对一个三元组来说，同连通块点对数量只有三种可能：

- 三个点在三个不同块：数量 0，合法；
- 两个点在同一个块：数量 1，不合法；
- 三个点都在同一个块：数量 3，不合法。

因此在模 2 意义下：

$$[\text{合法}] = 1 \oplus [\text{同块点对数量的奇偶性}]$$

于是固定 x 的非退化贡献奇偶性为：

$$T \bmod 2 \oplus \sum_i \binom{c_i}{2} \bmod 2$$

其中 c_i 是连通块 S_i 中 **非零点的数量**。

为什么只数非零点？

因为非退化三元组要求三个点都非零。任意两个不同非零点 a, b 唯一确定第三个点：

$$c = a \oplus b$$

并且 c 也非零且不同于 a, b 。

所以每个同块非零点对都唯一对应一个 XOR 零三元组。

非退化三元组总数

非零点共有 $N - 1$ 个。

任意两个不同非零点 a, b 决定第三个点 $a \oplus b$ ，每个三元组会被它的三条边数到三次。

所以：

$$T = \frac{\binom{N-1}{2}}{3} = \frac{(N-1)(N-2)}{6}$$

最终公式

令删掉点 x 后各连通块中的非零点数量为：

$$c_1, c_2, \dots, c_k$$

那么点 x 被异或进答案的奇偶性为：

$$p_x = [x \neq 0] \oplus \left(\frac{(N-1)(N-2)}{6} \bmod 2 \right) \oplus \bigoplus_i \left(\binom{c_i}{2} \bmod 2 \right)$$

最后答案就是：

$$\bigoplus_{p_x=1} x$$

如何在线性时间求所有 c_i

把树以 0 为根。

令：

$$sz_x$$

表示点 x 的子树中非零点的数量。

对于点 x ，删掉它后：

- 每个儿子 y 对应一个连通块，非零点数 sz_y ；
- 父亲方向对应一个连通块，非零点数：

$$N - 1 - sz_x$$

所以可以一次 DFS 求出所有 sz ，再扫一遍所有边即可。

3. Trick 与陷阱

Trick 点拨

核心 Trick 是：

把 $\text{lca}_{u \oplus v}(u, v)$ 转成三点 median，然后把点对重组成 XOR 为零的三元组。

另一个关键是：

只关心异或答案，所以只需要统计每个点作为答案出现次数的奇偶性。

于是复杂的 XOR 约束被转化为连通块中非零点数量的组合数奇偶性。

易错点

1. 必须单独处理 $u = 0$ 的退化情况。
每个非零点 x 都会因为点对 $(0, x)$ 额外贡献一次 x 。
2. 连通块大小要统计非零点数量，不是所有点数量。
因为非退化 XOR 零三元组不包含 0。
3. $\binom{c}{2} \bmod 2$ 可以写成：

$$\left\lfloor \frac{c}{2} \right\rfloor \bmod 2$$

即代码里：

```
(c >> 1) & 1
```

4. 不要递归 DFS。
 N 最大超过两百万，递归容易爆栈。使用迭代 DFS / BFS。

4. 代码实现 C++23

```
#include <bits/stdc++.h>

using namespace std;

using i64 = long long;

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);

    int T;
    cin >> T;

    while (T--) {
        int n;
        cin >> n;

        int N = 1 << n;

        vector<int> head(N, -1);
        vector<int> to(2 * (N - 1)), nxt(2 * (N - 1));

        int ec = 0;
        auto add = [&](int u, int v) {
            to[ec] = v;
            nxt[ec] = head[u];
            head[u] = ec++;
        };

        for (int i = 0; i < N - 1; i++) {
            int u, v;
```

```

        cin >> u >> v;
        add(u, v);
        add(v, u);
    }

    vector<int> par(N, -1), ord;
    ord.reserve(N);

    par[0] = -2;
    ord.push_back(0);

    for (int i = 0; i < N; i++) {
        int x = ord[i];
        for (int e = head[x]; e != -1; e = nxt[e]) {
            int y = to[e];
            if (y == par[x]) {
                continue;
            }
            par[y] = x;
            ord.push_back(y);
        }
    }

    vector<int> sz(N);

    for (int i = 0; i < N; i++) {
        sz[i] = ord[i] != 0;
    }

    for (int i = N - 1; i > 0; i--) {
        int x = ord[i];
        sz[par[x]] += sz[x];
    }

    auto c2 = [&](int x) -> int {
        return (x >> 1) & 1;
    };

    int all = ((i64)(N - 1) * (N - 2) / 6) & 1;

    int ans = 0;

    for (int x = 0; x < N; x++) {
        int p = (x != 0) ^ all;

        p ^= c2(N - 1 - sz[x]);

        for (int e = head[x]; e != -1; e = nxt[e]) {
            int y = to[e];
            if (par[y] == x) {
                p ^= c2(sz[y]);
            }
        }
    }

```

```
        if (p) {
            ans ^= x;
        }
    }

    cout << ans << '\n';
}

return 0;
}
```

5. 总结与启示

难度评分

预估 Codeforces Rating: **2600 ~ 2800**。

这题难点不在代码，而在把换根 LCA、XOR 点对、三元组中位点、奇偶计数串起来。

核心启示

当答案是 XOR 时，不要急着算具体次数；先把问题转化为“出现次数的奇偶性”。

同类习题推荐

1. **Codeforces 766E - Mahmoud and a xor trip**

树上路径与 XOR 贡献的经典题。

2. **AtCoder ABC201 E - Xor Distances**

按位拆贡献，理解树上 XOR 统计的好题。

3. **Codeforces 1709E - XOR Tree**

树结构与 XOR 性质结合，适合练习类似的转化思维。